

分治法答题模板

1 分治法基础

1.1 核心理想

将问题分解为若干个规模较小的相同问题，递归解决这些子问题，最后将子问题的解合并得到原问题的解。

1.2 基本步骤

1. 分解：将原问题分解为若干个规模较小的子问题
2. 解决：递归地求解各子问题
3. 合并：将子问题的解合并成原问题的解

1.3 适用条件

1. 问题可分：问题能分解为若干个规模较小的相同问题
2. 子问题独立：各子问题之间相互独立，不包含公共子问题
3. 可合并：子问题的解可以合并为原问题的解

1.4 与动态规划的区别

- 分治法：各子问题独立，递归求解，可能重复计算
- 动态规划：子问题重叠，通过记忆化或表格避免重复计算
- 选择标准：子问题独立用分治，子问题重叠用 DP

2 排序算法

2.1 归并排序

思想：分治法的典型应用，将数组分成两半分别排序，合并。

复杂度：时间 $O(n \log n)$ ，空间 $O(n)$ 。

稳定：稳定排序。

```
1 def merge_sort(arr, l, r):
2     if l >= r: return
3     mid = (l + r) // 2
4     merge_sort(arr, l, mid)
5     merge_sort(arr, mid + 1, r)
6     merge(arr, l, mid, r)
```

2.2 快速排序

思想：选择 pivot，将数组分为小于和大于 pivot 的两部分。

复杂度：平均 $O(n \log n)$ ，最坏 $O(n^2)$ 。

Partition：双指针或单指针扫描。

```
1 def quick_sort(arr, l, r):
2     if l >= r: return
3     p = partition(arr, l, r)
4     quick_sort(arr, l, p - 1)
5     quick_sort(arr, p + 1, r)
```

优化：随机选择 pivot 或三数取中，避免最坏情况。

3 查找问题

3.1 二分查找

前提：有序数组。复杂度： $O(\log n)$ 。

整数二分：注意边界条件，防止死循环。

```
1 def binary_search(arr, target):
2     l, r = 0, len(arr) - 1
3     while l <= r:
```

```
4         mid = (l + r) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             l = mid + 1
9         else:
10            r = mid - 1
11    return -1
```

3.2 快速选择（找第 k 小）

思想：基于快速排序的 partition，只递归一侧。

平均复杂度： $O(n)$ ，最坏 $O(n^2)$ 。

BFPRT 算法：最坏 $O(n)$ ，通过中位数的中位数选择 pivot。

```
1 def quick_select(arr, l, r, k):
2     if l >= r: return arr[l]
3     p = partition(arr, l, r)
4     if p == k:
5         return arr[p]
6     elif p > k:
7         return quick_select(arr, l, p - 1, k)
8     else:
9         return quick_select(arr, p + 1, r, k)
```

4 数学问题

4.1 大整数乘法（Karatsuba）

思想：将 n 位数字分成两个 $n/2$ 位数字。

公式： $x \cdot y = ac \cdot 10^n + [(a + b)(c + d) - ac - bd] \cdot 10^{n/2} + bd$ 。

复杂度： $O(n^{\log_2 3}) \approx O(n^{1.585})$ （朴素算法 $O(n^2)$ ）。

4.2 Strassen 矩阵乘法

思想：分治乘法，减少递归式中的乘法次数。

复杂度： $O(n^{\log_2 7}) \approx O(n^{2.807})$ （朴素算法 $O(n^3)$ ）。

关键：将 8 次乘法减少到 7 次，增加加减法次数。

5 几何问题

5.1 最接近点对

暴力： $O(n^2)$ 枚举所有点对。

分治解法：

1. 按 x 坐标排序，分成左右两半递归求解
2. 合并：检查跨越中线的点对（只需检查 strip 内 6 个点）

复杂度： $O(n \log^2 n)$ 或预处理排序后 $O(n \log n)$ 。

```
1 def closest_pair(points):
2     if len(points) <= 3:
3         return brute_force(points)
4     mid = len(points) // 2
5     left = points[:mid]
6     right = points[mid:]
7     d_left = closest_pair(left)
8     d_right = closest_pair(right)
9     d = min(d_left, d_right)
10    # 检查跨越中线的点对
```

```
11 return min(d, strip_closest(points, mid, d))
```

6 组合问题

6.1 棋盘覆盖

问题: $2^k \times 2^k$ 棋盘, 一个特殊方格, 用 L 型骨牌覆盖。

思想: 分割为 4 个 $2^{k-1} \times 2^{k-1}$ 子棋盘, 递归覆盖。

复杂度: $T(k) = 4T(k-1) + O(1) = O(4^k)$ 。

骨牌数: $(4^k - 1)/3$ 。

6.2 循环赛日程表

问题: n 个选手 (n 为 2 的幂), 每人每天比赛, 共 $n-1$ 天。

思想: 分成两半, 分别安排, 再交叉安排。

递归关系: 2^k 个选手的日程表可由 2^{k-1} 个选手的日程表构造。

7 分治法核心模式

7.1 标准模板

```
1 def divide_and_conquer(problem):
2     # 递归终止条件
3     if is_small(problem):
4         return solve_directly(problem)
5
6     # 分解: 将问题分解为若干子问题
7     subproblems = divide(problem)
8
9     # 解决: 递归解决各子问题
10    subresults = []
11    for sub in subproblems:
12        subresults.append(divide_and_conquer(sub))
13
14    # 合并: 将子问题的解合并
15    return combine(subresults)
```

7.2 常见递推关系

- 二分分治: $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$
- 归并排序: $T(n) = 2T(n/2) + O(n)$
- 二分查找: $T(n) = T(n/2) + O(1) \Rightarrow O(\log n)$
- 快速排序: $T(n) = 2T(n/2) + O(n)$ (平均)
- Karatsuba: $T(n) = 3T(n/2) + O(n) \Rightarrow O(n^{\log_2 3})$
- Strassen: $T(n) = 7T(n/2) + O(n^2) \Rightarrow O(n^{\log_2 7})$

7.3 主定理 (Master Theorem)

对于 $T(n) = aT(n/b) + f(n)$:

- 若 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$
- 若 $f(n) = \Theta(n^{\log_b a} \log^k n)$, 则 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$, 则 $T(n) = \Theta(f(n))$